

# Real-World Project

## Create Project

1. Go to [Spring Initializr](#).
2. Select **Maven Project**, **Java**, and **Spring Boot version** (latest stable).
3. Fill in the project metadata:
  - **Group:** `com.nbicocchi`
  - **Artifact:** `product-service-no-db`
4. Add the dependency **Spring Web**.
5. Click **Generate** to download the project as a `.zip`.
6. Extract and open the project in your IDE (e.g., IntelliJ IDEA, VS Code).

**Note:** *Spring Web keeps the app running and allows us to expose endpoints later.*

| Project                                                                      | Language                                                           |
|------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <input type="radio"/> Gradle - Groovy                                        | <input checked="" type="radio"/> Java <input type="radio"/> Kotlin |
| <input type="radio"/> Gradle - Kotlin <input checked="" type="radio"/> Maven | <input type="radio"/> Groovy                                       |

  
**Spring Boot**  
☐ 3.5.0 (SNAPSHOT) ☐ 3.5.0 (M1) ☐ 3.4.3 (SNAPSHOT)  
☒ 3.4.2 ☐ 3.3.10 (SNAPSHOT) ☐ 3.3.9  
  
**Project Metadata**

|              |                                                                                       |
|--------------|---------------------------------------------------------------------------------------|
| Group        | <code>com.nbicocchi</code>                                                            |
| Artifact     | <code>product-service-no-db</code>                                                    |
| Name         | <code>product-service-no-db</code>                                                    |
| Description  | Demo project for Spring Boot                                                          |
| Package name | <code>com.nbicocchi.product-service-no-db</code>                                      |
| Packaging    | <input checked="" type="radio"/> Jar <input type="radio"/> War                        |
| Java         | <input type="radio"/> 23 <input type="radio"/> 21 <input checked="" type="radio"/> 17 |

### Dependencies

ADD DEPENDENCIES... CTRL + B

#### Spring Web WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

## Persistence Layer

We'll create a simple [persistence layer](#) under the package `com.nbicocchi.product.persistence.model`, by adding a `Product` class.

```
@AllArgsConstructor
@NoArgsConstructor
@RequiredArgsConstructor
@Data
@Entity
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NonNull @EqualsAndHashCode.Include private String uuid;
    @NonNull private String name;
    @NonNull private Double weight;
}
```

This class has 3 key attributes: `uuid`, `name` and `weight`.

Next, we'll add the [repository](#) under the package *com.nbicocchi.product.persistence.repository*:

```
@Repository
public interface ProductRepository extends CrudRepository<Product, Long> {
    Optional<Product> findByUuid(String name);
}
```

## Service Layer

Moving on to the [service layer](#), we'll add a similar service under the package *com.nbicocchi.product.service*:

```
@Service
public class ProductService {
    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public Optional<Product> findByUuid(String uuid) {
        return productRepository.findByUuid(uuid);
    }

    public Iterable<Product> findAll() {
        return productRepository.findAll();
    }

    public Product save(Product product) {
        return productRepository.save(product);
    }

    public void delete(Product product) {
        productRepository.delete(product);
    }
}
```

## Presentation Layer

Moving on to the [presentation layer](#), we'll add a similar service interface under the package *com.nbicocchi.controller*:

```
@RestController
@RequestMapping("/products")
public class ProductController {
    ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping("/{uuid}")
    public Product findByUuid(@PathVariable String uuid) {
        return productService.findByUuid(uuid).orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));
    }

    @GetMapping
    public Iterable<Product> findAll() {
        return productService.findAll();
    }

    @PostMapping
    public Product create(@RequestBody Product product) {
        return productService.save(product);
    }
}
```

```

@PutMapping("/{uuid}")
public Product update(@PathVariable String uuid, @RequestBody Product product) {
    Optional<Product> optionalProject = productService.findByUuid(uuid);
    optionalProject.orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));
    product.setId(optionalProject.get().getId());
    return productService.save(product);
}

@DeleteMapping("/{uuid}")
public void delete(@PathVariable String uuid) {
    Optional<Product> optionalProject = productService.findByUuid(uuid);
    optionalProject.orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));
    productService.delete(optionalProject.get());
}
}

```

## ApplicationRunner and CommandLineRunner Interfaces

In Spring Boot, *CommandLineRunner* and *ApplicationRunner* are two interfaces that allow you to execute code when a Spring Boot application starts. They are typically used to perform some initialization or setup tasks before the application starts processing requests. Both interfaces have a single run method that you need to implement.

```

@Log
@SpringBootTest
public class App implements ApplicationRunner {
    ProductRepository productRepository;

    public App(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public static void main(final String... args) {
        SpringApplication.run(App.class, args);
    }

    @Override
    public void run(ApplicationArguments args) {
        productRepository.save(new Product("171f5df0-b213-4a40-8ae6-fe82239ab660", "Laptop", 2.2));
        productRepository.save(new Product("f89b6577-3705-414f-8b01-41c091abb5e0", "Bike", 5.5));
        productRepository.save(new Product("b1f4748a-f3cd-4fc3-be58-38316afe1574", "Shirt", 0.2));

        Iterable<Product> products = productRepository.findAll();
        for (Product product : products) {
            log.info(product.toString());
        }
    }
}

```

## Testing the Product microservice

See all products

```
curl -X GET http://localhost:7001/products | jq
```

```
[
  {
    "id": 163814,
    "uuid": "171f5df0-b213-4a40-8ae6-fe82239ab660",
    "name": "Laptop",
    "weight": 2.2
  },

```

```
{
  "id": 269752,
  "uuid": "f89b6577-3705-414f-8b01-41c091abb5e0",
  "name": "Bike",
  "weight": 5.5
},
{
  "id": 328487,
  "uuid": "b1f4748a-f3cd-4fc3-be58-38316afe1574",
  "name": "Shirt",
  "weight": 0.2
}
]
```

See one product

```
curl -X GET http://localhost:7001/products/171f5df0-b213-4a40-8ae6-fe82239ab660 | jq
```

```
{
  "id": 833124,
  "uuid": "171f5df0-b213-4a40-8ae6-fe82239ab660",
  "name": "Laptop",
  "weight": 2.2
}
```

Add a product

```
curl -X POST http://localhost:7001/products -H "Content-Type: application/json" -d '{
  "uuid": "b1f4748a-0000-4fc3-be58-38316afe1574",
  "name": "Puppet",
  "weight": 0.2
}'
```

```
{"id":777523,"uuid":"b1f4748a-0000-4fc3-be58-38316afe1574","name":"Puppet","weight":0.2}
```

Update a product

```
curl -X PUT http://localhost:7001/products/b1f4748a-0000-4fc3-be58-38316afe1574 -H "Content-Type: application/json" -d '{
  "uuid": "b1f4748a-0000-4fc3-be58-38316afe1574",
  "name": "Puppet, but nicer",
  "weight": 0.3
}'
```

```
{"id":777523,"uuid":"b1f4748a-0000-4fc3-be58-38316afe1574","name":"Puppet, but nicer","weight":0.3}
```

Delete a product

```
curl -X DELETE http://localhost:7001/products/b1f4748a-f3cd-4fc3-be58-38316afe1574 | jq
```

## Resources